

Код-ревью: RTSP Gateway

Панель управления камерами, публичные ссылки на просмотр, PTZ

Ветка `main` · коммит `58d3196` · 7 сентября 2026 г.

Общая оценка

Это заметно более зрелый код, чем встречается в проектах такого размера. Архитектура выдержана: PostgreSQL — единственный источник истины, MediaMTX держит только производное состояние, реконсильатор восстанавливает его после любого перезапуска. Границы доверия расставлены осознанно и в одном месте: SSRF-проверка живёт в `media/ssrf.py` и переиспользуется и камерами, и PTZ — с явным комментарием, почему её нельзя копировать. Секреты (RTSP-URL, TOTP, пароль SMTP) шифруются `libsodium SecretBox`, токены ссылок хранятся только хешами, пароли — `argon2id` с параметрами выше дефолтных.

Отдельно стоит отметить культуру комментариев: почти каждое неочевидное решение объяснено — не «что делает код», а «почему именно так и что сломается иначе». Это редкость, и она заметно снижает стоимость сопровождения. CSP с `per-response nonce` без `unsafe-inline`, серверные сессии в Redis с ротацией идентификатора при смене привилегий, защита TOTP от повторного использования кода, единая формулировка отказов, чтобы формы не превращались в справочник учётных записей, — всё это сделано правильно и без напоминаний.

Найденные проблемы не отменяют этой оценки. Ни одна из них не является критической: удалённого выполнения кода, SQL-инъекций, XSS или утечки секретов не обнаружено. Основная часть замечаний — это несколько несогласованностей между модулями (проверка есть на одном пути и отсутствует на соседнем), одна функционально нерабочая возможность (встраивание в чужой сайт) и набор оптимизаций, которые станут заметны при росте нагрузки.

Результаты автоматических проверок

Проверка	Команда	Результат
Тесты	<code>pytest -q</code>	435 passed, 96 c
Линтер	<code>ruff check app alembic tests</code>	All checks passed
Типы	<code>mypy app</code>	21 ошибка в 8 файлах — см. K-1
Объём	<code>app / tests / шаблоны</code>	7 366 / 2 667 / 3 891 строк

Тестовое покрытие серьёзное: 435 тестов на 7,4 тыс. строк приложения, включая проверки Caddyfile, docker-compose, миграций и вёрстки. Это не формальность — тесты содержательные и проверяют именно те места, где легко ошибиться.

Сводка находок

ID	Находка	Категория	Важность
B-1	Перезагрузка страницы просмотра обходит проверку IP-списка, пароля и лимита зрителей	Безопасность	Средняя
B-2	Встраивание <code>/embed</code> на чужой сайт не работает: cookie помечена <code>SameSite=Lax</code>	Баг	Средняя
B-3	<code>ViewSession.last_seen_at</code> никогда не обновляется; таблица не чистится	Баг	Средняя
B-4	Caddy доверяет <code>X-Forwarded-For</code> из частных сетей — подмена IP в режиме «по IP»	Безопасность	Средняя

ID	Находка	Категория	Важность
B-5	Ограничение размера ответа камеры в PTZ-драйверах не работает	Безопасность	Низкая
B-6	Снятие блокировки PTZ неатомарно (GET + DELETE)	Безопасность	Низкая
B-7	ValueError на подделанном поле формы превращается в 500	Баг	Низкая
B-8	Перевыпуск ссылки молча снимает отзыв	Баг	Низкая
B-9	allowed_cidrs сохраняются без проверки и молча игнорируются	Баг	Низкая
B-10	Нет защиты от отключения или понижения последнего администратора	Баг	Низкая
B-11	Гонка в счётчике rate-limit: окно может съезжать вперёд	Баг	Низкая
B-12	refresh_statuses не фильтрует камеры по node_id, в отличие от desired_state	Баг	Низкая
P-1	N+1 обращений к Redis при работе со списком сессий пользователя	Производительность	Средняя
P-2	Перебор кодов восстановления — до 10 проверок argon2 на одну попытку	Производительность	Низкая
P-3	Пагинация журнала аудита через OFFSET	Производительность	Низкая
P-4	sessions.touch() на каждом запросе панели	Производительность	Инфо
K-1	туру строгий в конфиге, но не в CI; накопился 21 разрыв типов	Качество	Средняя
K-2	Проверки безопасности в CI никогда не роняют сборку	Качество	Средняя
K-3	Мёртвый код и неиспользуемые поля	Качество	Низкая

1. Качество кода и соответствие практикам Python

Код написан на современном Python 3.12 и пользуется его возможностями по делу: `from __future__ import annotations` везде, `type IPNetwork = ...` (PEP 695), `StrEnum`, `dataclass(frozen=True, slots=True)` для неизменяемых значений, `Mapped[...]` в моделях SQLAlchemy 2.0. Асинхронность выдержана честно: единственное блокирующее место — `smtpplib` — вынесено в `asyncio.to_thread` с объяснением, почему не взята асинхронная библиотека.

Что сделано особенно хорошо

- **Разделение ответственности.** `probe.py` отвечает на вопрос «какие кодеки», `diagnose.py` — на вопрос «почему чёрный экран». Это разные задачи, и их не свалили в один модуль.
- **Драйверы PTZ за протоколом.** Наружу торчат ровно два действия — «двигайся» и «стой», весь вендорский разноразмерный заперт в четырёх драйверах, разница в нумерации каналов у Dahua спрятана внутри драйвера, а не вынесена в интерфейс оператора.
- **Обработка ошибок соразмерна цене.** Битый `argon2`-хеш не роняет форму входа, нерасшифровываемый пароль SMTP не роняет отправку, диагностический зонд ловит `Exception` целиком — и в каждом случае это объяснено комментарием.
- **Единый глобальный обработчик ошибок** отдаёт страницу с кодом запроса вместо «Internal Server Error», а для `/internal/` — голый статус, потому что HTML там читать некому.
- **Миграции не накатываются при старте** — осознанное решение с объяснением, а расхождение схемы называется вслух в логе и в `/readyz`.

К-1 · туру настроен строго, но не запускается в CI

В `pyproject.toml` стоит `[tool.mypy] strict = true`, но `.github/workflows/ci.yml` запускает только `ruff` и `pytest`. Проверка типов не выполняется никогда, и за это время накопился 21 разрыв.

Файл	Характер	Оценка
<code>auth/sessions.py</code> , <code>internal/authz.py</code> , <code>web/public_views.py</code>	<code>Awaitable[T]</code> <code>T</code> в <code>await</code> — особенность типизации <code>redis-py</code>	Шум, но глушит настоящие ошибки
<code>web/panel_views.py:288</code>	<code>**dict[str, object]</code> подставляется в параметр <code>status_code: int</code>	Настоящий разрыв контракта
<code>media/diagnose.py:438-439</code>	итерация и <code>int()</code> по значению типа <code>object</code> ; <code>type: ignore</code> не покрывает код ошибки	Устаревшие подавления
<code>auth/deps.py:85</code> , <code>web/panel_views.py:60</code>	функции без аннотации возврата	Мелочь, чинится за минуту

Как чинить. Добавить шаг `туру app` в CI. Разрывы вокруг `redis-py` закрываются одной строкой в конфиге (`[[tool.mypy.overrides]] module = "redis.*"`), оставшиеся четыре — точечными правками. Пока проверка не в CI, число будет только расти.

К-2 · CI сообщает о проблемах безопасности, но не останавливается на них

```
- name: Уязвимости в зависимостях
  run: pip-audit --strict --desc
  continue-on-error: true          # ← результат игнорируется

- uses: aquasecurity/trivy-action@master
  with:
    severity: HIGH,CRITICAL
    exit-code: "0"                # ← находки не роняют сборку

shellcheck --severity=warning install.sh ... || true
```

Все три проверки безопасности носят справочный характер: уязвимая зависимость или HIGH-находка в образе пройдут в `main` незамеченными, если никто не откроет лог сборки. Это разумно как стартовая настройка, чтобы не блокировать работу чужими ложными срабатываниями, но со временем превращается в шум, который никто не читает. Практичный компромисс — оставить `continue-on-error` для ежедневных прогонов, но сделать проверки блокирующими на релизном теге, и завести файл исключений для заведомо неприменимых CVE.

К-3 · Мёртвый код и неиспользуемые поля

Место	Что
<code>ptz/transport.py:67</code>	<code>forget_auth()</code> определена «на случай смены пароля камеры», но не вызывается ниоткуда. При смене учётных данных PTZ кэш <code>DigestAuth</code> в памяти процесса останется старым — <code>_apply_ptz_form</code> сбрасывает <code>ptz_meta</code> , но не этот кэш.
<code>models.py:391</code>	<code>ViewSession.session_key</code> генерируется и индексируется как <code>unique</code> , но по нему никогда ничего не ищут (см. В-3).
<code>ptz/transport.py:105</code>	<code>ensure_digest()</code> вызывается только в <code>identify()</code> , но не в <code>move()/stop()</code> . Утечки пароля это не даёт (<code>httpx.DigestAuth</code> не отвечает на <code>Basic</code> -вызов), но асимметрия выглядит как забытое место.

2. Потенциальные баги и необработанные граничные случаи

В-1 Перезагрузка страницы просмотра обходит IP-список, пароль и лимит зрителей

Средняя

backend/app/web/public_views.py:169-197, :136-144

Что не так. На «быстром пути» перезагрузки страницы без токена вызывается `_open_stream()` напрямую, минуя `_resolve_link()` — а вместе с ним и все проверки ссылки, кроме факта наличия гранта в Redis:

```
link = await db.scalar(select(ShareLink).where(ShareLink.slug == slug))
camera = await db.get(Camera, link.camera_id) if link is not None else None

if link is not None and camera is not None and not token:
    if await _already_granted(request, camera):
        return await _open_stream(request, db, link, camera, embed=embed)
    # ни ip_allowed(), ни password_hash, ни expires_at, ни max_concurrent
```

Почему это важно. Проверка `ip_allowed()` выполняется только в `_resolve_link()`. На пути media-запросов её тоже нет: `authz.link_is_valid()` сверяет только `revoked_at`, `expires_at` и то, включена ли камера. Значит зритель, однажды открывший ссылку с разрешённого адреса, дальше смотрит её с **любого адреса**: cookie переустанавливается на новые 60 минут при каждой перезагрузке, а грант в Redis продлевается `authz._allow()` на каждом сегменте HLS. То же касается пароля ссылки и `max_concurrent`. Дополнительно `_already_granted()` ищет грант по `camera.mtx_path`, а не по идентификатору ссылки — поэтому грант, выданный ссылкой А (или просмотром оператора из панели), открывает страницу под slug ссылки В на ту же камеру. Показательно, что соседний обработчик `view_ptz()` (`public_views.py:237`) `ip_allowed()` вызывает — то есть это не сознательное послабление, а пропуск.

Как чинить. Перенести проверки, не зависящие от токена, в общий путь. Минимально: вызывать `ip_allowed()` и сверять `expires_at` / `revoked_at` и в ветке перезагрузки, а `_already_granted()` сверять с `str(link.id)`, а не с фактом наличия ключа. Правильнее — добавить проверку списка адресов в `link_is_valid()`, чтобы ограничение работало и на media-запросах, а не только на HTML-странице.

В-2 Встраивание /embed на сторонний сайт не может работать

Средняя

backend/app/middleware.py:22,97 · backend/app/web/public_views.py:124-132 · docs/security.md:112

Что не так. Для `/embed/*` специально ослаблена политика встраивания — `frame-ancestors *` и снятый `X-Frame-Options`. Но cookie доступа `rtspgw_view` выдаётся с `samesite="lax"`.

Почему это важно. Cookie с `SameSite=Lax` браузер отправляет только при навигации верхнего уровня. Внутри `iframe` на чужом домене запросы за `/hls/...` и `/whip/...` — межсайтовые подзапросы, cookie в них не уходит, `forward_auth` получает запрос без неё и отвечает 403 (`reason=no_view_cookie`). Зритель видит чёрный экран. То есть возможность заявлена в документации и в коде, но на чужом сайте не работает ни при каких условиях; работает только встраивание на своём же домене — для которого послабление CSP не требуется.

Как чинить. Определиться с назначением. Если встраивание в чужие страницы нужно — выдавать cookie просмотра с `SameSite=None`; `Secure` (защита от CSRF на публичном пульте уже построена на `X-Requested-With` и `JSON-preflight`, а не на `SameSite`, поэтому ослабление не откроет новых путей) и учесть, что Safari и Firefox режут сторонние cookie по умолчанию. Если не нужно — вернуть `frame-ancestors 'self'` и убрать обещание из `docs/security.md`.

В-3 ViewSession живёт ровно 5 минут и никогда не удаляется

Средняя

backend/app/models.py:382–398 · backend/app/worker.py:112–135 · backend/app/main.py:155–158

Что не так. Поле `ViewSession.last_seen_at` заполняется один раз через `server_default` при вставке и больше не обновляется нигде — поиск по всему `app/` даёт единственное чтение в `_cleanup_cycle`.

Почему это важно. Из этого следуют три расхождения с задуманным. Первое: каждый сеанс принудительно закрывается через 5 минут после начала, независимо от того, смотрит зритель или нет. Второе: метрика `rtspgw_active_viewers` (`count(*) ... WHERE ended_at IS NULL`) показывает не число зрителей, а число открытий страницы за последние минуты — по ней нельзя строить ни график, ни алерт. Третье, и самое неприятное в долгой перспективе: `_cleanup_cycle` удаляет только `Invitation`, а строки `view_sessions` копятся вечно — по строке на каждое открытие и каждую перезагрузку публичной ссылки. Это самая быстрорастущая таблица в схеме, и ничто её не подрезает.

Как чинить. Либо обновлять `last_seen_at` — естественное место есть, это `authz._allow()`, который и так вызывается на каждом `media`-запросе (по `session_key`, ради которого заведён `unique`-индекс), либо честно признать таблицу журналом просмотров: убрать `last_seen_at` и `session_key`, считать метрику зрителей из Redis (там уже есть `count_viewers`) и добавить в `_cleanup_cycle` удаление строк старше срока хранения — по образцу `INVITE_RETENTION_DAYS`.

В-7 Подделанное значение поля формы даёт 500 вместо понятной ошибки

Низкая

backend/app/web/panel_views.py:242, :334, :164 · backend/app/web/admin_views.py:222

Что не так. `StreamProfile(profile)` и `Role(role)` вызываются напрямую от значения поля формы. Неизвестное значение поднимает `ValueError`, который доходит до глобального обработчика и превращается в страницу «Внутренняя ошибка сервиса».

Почему это важно. Показательно, что рядом это сделано правильно: `camera_set_profile` (`panel_views.py:482`) оборачивает тот же вызов в `try/except` и отвечает `Forbidden`, а `user_invite` (`admin_views.py:128`) — то же самое для `Role`. То есть автор знает про этот случай, просто в четырёх местах его пропустил. Эксплуатации здесь нет — нужна и сессия, и валидный CSRF-токен, — но в журнале появляется ложная пятисотка, а оператор получает нечитаемую ошибку.

Как чинить. Вынести разбор в общий помощник (по образцу `_int_or` и `_positive_int`, которые в проекте уже есть) и использовать его во всех четырёх местах.

В-8 Перевыпуск токена ссылки молча снимает отзывы

Низкая

backend/app/web/panel_views.py:829–831

Что не так. `link_rotate()` выполняет `link.revoked_at = None` без всяких условий. Ранее отозванная ссылка возвращается в строй, причём в интерфейсе это никак не отмечено — кнопка называется «перевыпустить адрес».

Почему это важно. Отзыв ссылки — операция с намерением «доступ по этому адресу закрыт». Восстановление его нажатием соседней кнопки — не то, чего ждёт оператор, и такое действие обычно делают явным. В журнале аудита при этом остаётся только `link.rotated`, без указания, что ссылка была отозвана.

Как чинить. Либо не трогать `revoked_at` при перевыпуске (отозванная остаётся отозванной), либо спрашивать подтверждение и записывать факт восстановления в `meta` события аудита.

В-9 Список разрешённых подсетей сохраняется без проверки

Низкая

backend/app/web/panel_views.py:781 · backend/app/internal/authz.py:140-153

Что не так. `allowed_cidrs=[c.strip() for c in allowed_cidrs.split(",") if c.strip()]` — строки кладутся в базу как есть. `ip_allowed()` при разборе неверного CIDR делает `continue`, то есть молча его пропускает.

Почему это важно. Поведение при опечатке безопасно (список непустой, ни один адрес не совпал — ссылка перестаёт работать для всех), но необъяснимо для оператора: он видит «Доступ с этого IP-адреса запрещён владельцем ссылки» и не понимает, почему, потому что в форме он ошибку не получил. Есть и обратный случай: если ввести только разделители (, , или пробелы), список станет пустым и ограничение исчезнет вовсе — без единого сигнала.

Как чинить. Разбирать `ipaddress.ip_network(item, strict=False)` при сохранении и возвращать форму с ошибкой по образцу того, как это уже сделано для `CAMERA_HOST_ALLOWLIST` в `config.py:164-175`.

В-10 Последнего администратора можно отключить или понизить

Низкая

backend/app/web/admin_views.py:182-228

Что не так. `user_toggle()` и `user_role()` защищают только от действий над собой (`target.id == admin.id`). Двое администраторов могут по очереди отключить друг друга, либо один может понизить второго и потерять доступ вместе с ним, если его собственная учётка потом заблокируется по неудачным попыткам входа.

Почему это важно. Восстановление потребует доступа к серверу и `python -m app.cli create-admin`. Для сервиса, который ставят и забывают, это выход из строя панели администратора до приезда того, у кого есть SSH.

Как чинить. Перед отключением или понижением проверять, что останется хотя бы один активный администратор: `select count(*) from users where role='admin' and is_active`. Запасной путь через CLI при этом остаётся и никуда не девается.

В-11 Гонка в счётчике ограничения частоты

Низкая

backend/app/auth/ratelimit.py:50-69

Что не так. `INCR` и `TTL` идут одним конвейером, но `EXPIRE` ставится следующей командой, уже после разбора ответа. Два одновременных первых запроса оба увидят `ttl < 0` и оба вызовут `EXPIRE`, сдвигая окно вперёд.

Почему это важно. Практический эффект невелик: окно может съехать на доли секунды, лимит от этого становится чуть строже, а не мягче. Опаснее второй сценарий: если `EXPIRE` не выполнится (обрыв соединения с Redis ровно между двумя командами), ключ останется без TTL навсегда, а Redis настроен с `maxmemory-policy noeviction` — то есть никогда его не вытеснит. Пользователь или IP окажется заблокирован до ручного вмешательства.

Как чинить. Свести обе операции в один атомарный шаг: `SET key 0 EX window NX` перед `INCR` в одном конвейере, либо Lua-скрипт `INCR + EXPIRE NX`. Второй вариант заодно снимает и первую проблему.

backend/app/media/reconciler.py:44-50 против :131-137

Что не так. `desired_state()` отбирает камеры с `Camera.node_id == node_id`, а `refresh_statuses()` в соседней функции берёт **все** включённые камеры без этого условия.

Почему это важно. Пока узел один, разницы нет. Но `node_id` заведён в схеме именно как «задел на горизонтальное масштабирование», и в тот день, когда узлов станет два, второй узел начнёт считать чужие камеры сломанными (их путей нет в его MediaMTX), ставить им `status=offline`, накручивать `failure_streak` и на 5-м цикле пытаться пересоздать чужой путь у себя. Найти это будет тяжело: симптом «камеры мигают между `online` и `offline`» не указывает на источник.

Как чинить. Добавить тот же фильтр по `node_id` в `refresh_statuses()` — одна строка, которая сейчас ничего не меняет, но снимает целый класс будущих проблем.

3. Производительность

Тяжёлых ошибок вроде N+1 в SQL или синхронного ввода-вывода в цикле событий здесь нет. Запросы к базе выстроены аккуратно: счётчики ссылок берутся одним `GROUP BY`, а не запросом на камеру; метрики считаются на момент запроса, а не хранятся в памяти с синхронизацией между процессами; фоновые задачи разнесены по тикам, чтобы тяжёлое не крутилось каждые 15 секунд. Замечания ниже — про запас на рост, а не про текущие проблемы.

Р-1 N+1 обращений к Redis при работе со списком сессий

Средняя

backend/app/auth/sessions.py:194-218

Что не так. `list_for_user()` делает `SMEMBERS`, затем `HGETALL` отдельным круговым обменом на каждый идентификатор. `delete_all_for_user()` поверх этого делает ещё по два обмена на каждую сессию: `delete()` сам вызывает `load()`, чтобы узнать `user_id`, который вызывающему уже известен.

Почему это важно. Открытие `/profile` у человека с десятью сессиями — 11 последовательных обменов с Redis. Смена пароля, отключение пользователя администратором и «завершить остальные сеансы» стоят порядка 3N обменов. При Redis в соседнем контейнере это единицы миллисекунд, но при вынесении Redis на отдельный хост задержка умножается на N и становится заметной.

Как чинить. Собрать `HGETALL` в один конвейер (`pipeline()` уже используется в этом же модуле). В `delete_all_for_user()` добавить внутренний вариант удаления, принимающий уже известный `user_id`, — это убирает лишний `load()` на каждую сессию.

Р-2 Перебор кодов восстановления — до 10 проверок argon2 на одну попытку

Низкая

backend/app/web/auth_views.py:258–269 · backend/app/auth/passwords.py:12

Что не так. `_try_recovery_code()` перебирает все неиспользованные коды и для каждого выполняет полную проверку `argon2id` (`time_cost=3`, `memory_cost=64` МБ). Это неизбежно — коды хранятся хешами и искать их по значению нельзя, — но цена стоит проговорить.

Почему это важно. Каждый неверный код второго фактора при десяти живых кодах восстановления — это около 0,5–0,8 с процессорного времени и до 640 МБ последовательных выделений памяти. Лимит `TOTP_BY_SESSION` (6 попыток за 5 минут на сессию) ограничивает одну сессию, но считается по `session.sid` — а сессий с состоянием «пароль принят, жду код» можно завести много, зная лишь один валидный пароль. Это не дыра, а усилитель: одна учётная запись даёт заметно больше нагрузки, чем обычная попытка входа.

Как чинить. Добавить к коду восстановления короткий незашифрованный префикс-указатель (первые 5 символов до дефиса) и искать по нему, оставив `argon2` для одного кандидата вместо десяти. Как более простой шаг — ввести дополнительный лимит по `user_id`, а не только по `sid`.

Р-3 Пагинация журнала аудита через OFFSET

Низкая

backend/app/web/admin_views.py:257–287

Что не так. `query.offset(page * AUDIT_PAGE_SIZE).limit(...)`. PostgreSQL обязан прочитать и отбросить все пропускаемые строки, поэтому стоимость страницы растёт линейно с её номером.

Почему это важно. `audit_log` — таблица только на добавление, в которую пишется каждый вход, каждый просмотр по ссылке и каждый отказ. Через год работы в ней сотни тысяч строк, и переход на 500-ю страницу превращается в полноценное сканирование. Индекс по `created_at` есть и помогает сортировке, но не отбрасыванию.

Как чинить. Перейти на `keyset`-пагинацию: вместо номера страницы передавать курсор (`created_at`, `id`) последней показанной записи и добавлять `WHERE (created_at, id) < (:ts, :id)`. Стоимость страницы становится постоянной. Заодно стоит подумать о сроке хранения журнала.

Р-4 `sessions.touch()` на каждом запросе панели

Инфо

backend/app/auth/deps.py:78 · backend/app/auth/sessions.py:154–162

Что не так. `require_user()` продлевает скользящее окно на каждом обращении — четыре команды Redis (одним конвейером) на каждый запрос авторизованного пользователя, включая те, что ничего не меняют.

Почему это важно. Это не ошибка, а сознательная плата за то, что активный пользователь не выпадает по таймауту. Отмечено для полноты: если панель когда-нибудь начнёт опрашивать сервер часто (автообновление списка камер, длинный поллинг), эта запись станет самой частой операцией в системе — и она именно запись, а не чтение.

Как чинить. Если это когда-нибудь понадобится: продлевать не чаще чем раз в N секунд, сравнивая `last_seen` с текущим временем перед записью. Сейчас менять ничего не нужно.

4. Безопасность

Модель угроз в этом проекте продумана, и это видно по коду, а не только по документации. Ниже — сначала то, что проверено и оказалось сделано верно, затем находки. Отдельно отмечу: осознанные компромиссы, зафиксированные комментариями в коде (`verify=False` для самоподписанных сертификатов камер, DNS rebinding как остаточный риск, необязательный префикс в `_MEDIA_URI`, отсутствие сторожа PTZ, переживающего перезапуск), в находки не вынесены — это ровно то, о чём

просили не сообщать.

Проверено и признано корректным

Направление	Вывод
SQL-инъекции	Не найдено. Везде параметризованный SQLAlchemy; <code>text()</code> используется только с литеральными строками без подстановок.
XSS	Не найдено. Jinja2 с автоэкранированием, единственный <code> safe (profile_2fa.html:32)</code> применён к SVG, сгенерированному библиотекой на сервере. <code>innerHTML</code> в <code>panel.js</code> заполняется только ответом своего же эндпоинта. Инлайновых обработчиков нет, CSP их и не пропустит.
Открытый редирект	Проверено с обходными полезными нагрузками (<code>/\evil.com, /%09/evil.com</code>) — не воспроизводится: Starlette кодирует их в %5C и оставляет %09, поэтому браузер не сворачивает путь в протокол-относительный адрес. <code>safe_next()</code> достаточен, хотя опирается на это поведение неявно.
SSRF	Закрывает добросовестно и в одном месте: только <code>rtsp/rtsp</code> s, все адреса имени обязаны быть публичными, PTZ ходит через ту же функцию и только на <code>camera.host</code> , <code>XAddr</code> из ответа ONVIF урезается до пути, редиректы не разворачиваются, <code>trust_env=False</code> .
Инъекция аргументов в ffmpeg	Команда собирается списком и передаётся MediaMTX без shell; URL до этого проходит фильтр <code>_FORBIDDEN_RAW</code> , отсекающий пробел, кавычки, обратный слэш и управляющие символы. <code>ffprobe</code> и <code>ffmpeg</code> запускаются через <code>create_subprocess_exec</code> , не через shell.
Инъекция в заголовки писем	<code>_check_header()</code> отсекает <code>\r\n</code> во всех заголовках; <code>EMAIL_RE</code> намеренно использует <code>\A...\Z</code> , а не <code>^...\$</code> , чтобы адрес с хвостовым переводом строки не прошёл.
CSRF	Токен в серверной сессии, сверка через <code>compare_digest</code> , <code>SameSite=Lax</code> . Публичный пульт закрыт иначе — тремя независимыми условиями, и это обосновано в докстринге.
Хранение секретов	<code>XSalsa20-Poly1305</code> с версионированием формата, ключ читается на старте (битый роняет процесс сразу), в журнале аудита и в логах паролей нет, <code>strip_credentials()</code> применяется к выводу <code>ffprobe</code> и <code>ffmpeg</code> до записи в лог. <code>rotate-key</code> не забывает про PTZ-креды и TOTP.
Аутентификация	<code>argon2id</code> выше дефолтных параметров, хеш-заглушка для постоянного времени ответа, блокировка после 10 попыток, два независимых лимита частоты, ротация идентификатора сессии после входа и после второго фактора, защита TOTP от повторного использования кода, <code>needs_rehash</code> .

Находки

В-4 Caddy доверяет X-Forwarded-For из частных сетей

Средняя

Caddyfile:24-27 · backend/app/middleware.py:28-41

Что не так. В глобальном блоке стоит `trusted_proxies static private_ranges` вместе с `client_ip_headers X-Forwarded-For`. Caddy считает доверенным прокси любой источник из частных диапазонов и берёт IP клиента из присланного им заголовка.

Почему это важно. Для развёртывания на VPS с публичным адресом это безопасно — клиенты приходят с публичных адресов, и заголовок игнорируется. Но Caddyfile сам описывает второй поддерживаемый режим — «доступ по IP», без домена, — а он и раздел про `sslip.io` подразумевают в том числе установку в локальной сети. Там клиенты приходят с `192.168.x.x` или `10.x.x.x`, попадают в `private_ranges` и могут назвать любой собственный адрес. Это обходит `allowed_cidrs` у публичных ссылок, обнуляет лимиты частоты по IP (включая перебор пароля и формы восстановления) и подделывает адрес в журнале аудита — а журнал в `docs/security.md` заявлен как то, «что спросят первым делом, если ссылка утечёт».

Как чинить. Убрать `trusted_proxies` и `client_ip_headers` из конфигурации по умолчанию — Caddy сам является границей, и без них `{client_ip}` берётся из адреса соединения, то есть подделать его нельзя. Оставить в файле закомментированный блок с конкретной подсетью на случай, если перед Caddy когда-нибудь встанет балансировщик — как это уже описано в комментарии рядом.

В-5 Ограничение размера ответа камеры в PTZ-драйверах не работает

Низкая

backend/app/ptz/onvif.py:60-61,264 · backend/app/ptz/vendors.py:23-24,60,141,195

Что не так. Комментарий гласит: «Дальше этого тело ответа не читаем — защита от бесконечного потока», а код делает `response.text[:_MAX_BODY]`. Но `httpx` без `stream=True` вычитывает тело целиком в память ещё до обращения к `.text`. Срез отбрасывает уже загруженные данные, то есть защиты нет.

Почему это важно. Устройство по адресу камеры отвечает нам произвольно большим телом, и весь процесс `api` буферизует его в памяти. Достижимо это только для того, кто контролирует камеру или то, что отвечает на её HTTP-порту, — а адрес камеры вводит оператор, — поэтому важность низкая. Но контейнер работает с `read_only: true` и без ограничения памяти в `compose`, так что процесс убьёт OOM-killer вместе со всей панелью.

Как чинить. Либо перейти на `client.stream()` с чтением по частям и обрывом на `_MAX_BODY`, либо — проще и достаточно — проверять заголовок `Content-Length` и отклонять ответ до чтения тела. Заодно поправить комментарий, чтобы он не обещал того, чего нет. Ограничение памяти для контейнеров в `docker-compose.yml` — полезно независимо от этого.

В-6 Снятие блокировки PTZ неатомарно

Низкая

backend/app/ptz/lock.py:57-62, :44-54

Что не так. `release()` выполняет GET, сравнивает владельца и делает DELETE отдельной командой. Между ними ключ может истечь по TTL и быть захвачен другим зрителем — тогда DELETE снимет чужую блокировку. То же в `acquire()`: продление идёт как GET плюс EXPIRE.

Почему это важно. Это классический паттерн, ради которого в Redis существует сравнение-и-удаление на Lua. Последствие здесь мягкое: второй зритель на секунду теряет только что полученное управление камерой и увидит «Камерой управляет другой». Данные не утекают, ничего не ломается. Но окно узкое ровно потому, что TTL составляет 15 секунд, а команды идут раз в 700 мс — при изменении этих чисел вероятность вырастет.

Как чинить. Заменить на стандартный Lua-скрипт `if redis.call('get',KEYS[1])==ARGV[1] then return redis.call('del',KEYS[1]) end` и аналогичный для продления. Клиент `redis.asyncio` это умеет через `register_script()`.

Что делать в первую очередь

Порядок ниже — по соотношению «цена исправления к цене того, что оно предотвращает», а не строго по важности.

№	Действие	ID	Объём
1	Убрать <code>trusted_proxies private_ranges</code> из <code>Caddyfile</code>	B-4	Одна строка
2	Проверять IP-список и срок действия ссылки на пути перезагрузки; сверять грант с идентификатором ссылки	B-1	~15 строк
3	Добавить <code>муру app</code> в CI и закрыть 21 разрыв типов	K-1	Полдня
4	Решить судьбу <code>view_sessions</code> : обновлять <code>last_seen_at</code> либо чистить таблицу по сроку хранения	B-3	~20 строк
5	Определиться со встраиванием: <code>SameSite=None</code> либо отказ от <code>frame-ancestors *</code>	B-2	Решение + правка
6	Атомарные операции в Redis: блокировка PTZ и счётчик частоты	B-6, B-11	~20 строк
7	Единый разбор значений <code>enum</code> из форм; проверка <code>allowed_cidrs</code> ; защита последнего администратора	B-7, B-9, B-10	~40 строк
8	Конвейеризовать работу со списком сессий; <code>keyset</code> -пагинация журнала	P-1, P-3	~30 строк
9	Фильтр по <code>node_id</code> в <code>refresh_statuses</code> ; убрать мёртвый код	B-12, K-3	Одна строка + уборка

Резюме. Проблем, требующих немедленной остановки эксплуатации, нет. Код соответствует хорошим практикам Python и в большинстве спорных мест выбирает правильную сторону — причём с объяснением, почему. Основной систематический мотив всех находок один и тот же: проверка реализована на одном пути и пропущена на соседнем (IP-список есть в PTZ и нет в просмотре, разбор `enum` обёрнут в двух местах из шести, фильтр по узлу есть в одной функции и нет в соседней). Это лечится не переписыванием, а сведением дублирующихся проверок в одну точку — ровно тем приёмом, который автор уже применил к SSRF и подробно обосновал в комментариях к `assert_reachable_host()`.